

1. Дисциплина: Языки программирования Тема: Многопоточность в C++ ФИО: Самошкин Егор Вадимович Группа: СКБ-251
2. Постановка задачи

Необходимо реализовать код на языке **C++ (C++11 и выше)**, который обеспечивает обработку задач несколькими потоками. Для этого необходимо:

1. Реализовать потокобезопасную очередь задач TaskQueue .
2. Создать несколько рабочих потоков worker.
3. Потоки получают задания из очереди и обрабатывают из 1 секунду.
4. Использовать механизмы синхронизации стандартной библиотеки C++.
5. Корректно завершить работу потоков.
6. Реализация программы:

3.1 Заголовочные файлы:

#include - ввод-вывод

#include - Создание потоков

#include - работа с дин. массивом потоков

#include - для создания очереди

#include - для работы с мьютексом

#include <condition_variable> - для работы с condition_variable

#include - для таймера обработки

3.2 Класс TaskQueue:

Хранит очередь, мьютекс для очереди и condition_variable для контроля доступа потоков к данным и предотвращения гонок данных.

Метод push - принимает целое число - номер задачи, блокирует мьютекс, добавляет номер задачи и пробуждает один из ждущих потоков. Как только произойдет выход из этого куска кода, удалится старый Lock и мьютекс освободится (ввиду использования `std::unique_lock` `std::mutex`)

Метод pop - принимает целое число-номер задачи, блокирует мьютекс, если очередь пуста, мьютекс освобождается и поток засыпает. Иначе удаляет номер процесса и аналогично Push освобождает мьютекс. `std::mutex coutMutex`; - мьютекс для корректного вывода.

3.3 Функция worker:

Ждет задачу с помощью цикла и pop (Это НЕ busy-wait, т.к в случае отсутствия задач поток засыпает, а не постоянно крутит условие, строчка `cv.wait...` в `pop()` это обеспечивает). Здесь `conditional_variable` т.е `cv`, принимает `unique_lock` и условие продолжения. Если условие не

выполнено, сон продолжается до его наступления, причем все это время мьютекс свободен. Как только происходит `cv.notify_one` один из потоков пробуждается и условие гарантированно срабатывает тк перед `Notify` Происходит добавление задачи.

если на вход получает -1 - поток завершается (задач больше не будет), иначе ждет 1 секунду и выводит сообщение об обработанном процессе.

3.4 Основная функция main:

Создает очередь `queue` типа `TaskQueue`, создает динамический массив работников (потоков), их число $N = 4$ (например) по следующей схеме:

В очередь потоков добавляется объект с находящейся в нем функцией, ссылкой на очередь (для этого используется `ref`), и номером. затем очередь наполняется 20-ю процессами и 4-мя "-1" для каждого потока, чтобы в конце завершиться. Это гарантирует завершение т.к. каждый поток примет -1 ровно 1 раз и завершится, гарантировано что рано или поздно они ее примут.

В конце запускаем потоки с помощью `join` и ждем завершения.

7. Ссылка: <https://github.com/yegor1202/hse>

The screenshot shows a VS Code editor with the following content:

```

9  class TaskQueue {
22  int pop() {
24
25      cv.wait(lock, [this]() {
26          return !tasks.empty();
27      });
28
29      int task = tasks.front();
30      tasks.pop();
31      return task;
32  };
33  };
34
35  std::mutex coutMutex;
36

```

The terminal output shows the execution of the program with 4 workers processing 20 tasks each, followed by a final output of 19 tasks per worker:

```

yegor@EgorSamoshkin MINGW64 ~/Desktop/VSCode/hwModule (master)
$ /usr/bin/env c:\Users\yegor\vscode\extensions\ms-vscode.cpptools-1.30.5-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe --stdin=Microsoft-MIEngine-In-h
fnqzimb.stk --stdout=Microsoft-MIEngine-Out-1embiwei.lct --stderr=Microsoft-MIEngine-Error-s543btj1.slw --pid=Microsoft-MIEngine-Pid-5yflxzhk.gup --dbgExe=C:\Users\ya
ego\gcc\bin\gdb.exe --interpreter=mi
[Worker-3] обработал задачу 5
[Worker-2] обработал задачу 10
[Worker-4] обработал задачу 11
[Worker-1] обработал задачу 9
[Worker-3] обработал задачу 12
[Worker-2] обработал задачу 13
[Worker-1] обработал задачу 15
[Worker-4] обработал задачу 14
[Worker-3] обработал задачу 16
[Worker-2] обработал задачу 17
[Worker-1] обработал задачу 18
[Worker-3] обработал задачу 20
[Worker-4] обработал задачу 19
yegor@EgorSamoshkin MINGW64 ~/Desktop/VSCode/hwModule (master)
$

```

8. Выводы

Я изучил основы работы с многопоточностью в C++, понял как работает мьютекс, `condition_variable` и что это такое, реализовал в программе модель потокобезопасной очереди.